

HIP-008 — Arquitetura do App Móvel (Fase 2 · Etapa 2)

Status: Approved · **Architectural Baseline** · **Versão:** 1.0 · **Histórico:** v1.0 (2026-07-20) criação + reforço app-produto-principal. Sob ADR-000 · ARCH-002 · deriva de HIP-007. Decisão: ADR-006. **Premissa (rev.):** a escolha de tecnologia é uma **decisão estratégica da empresa**. Sob ARCH-002, o **app é o PRODUTO PRINCIPAL da SINTERA** (a web é interface complementar) e o backend é **API-first** (Mobile → API → Web). O MVP faz apenas auth + Apple Health + Health Connect + sync, mas a arquitetura já nasce de produto completo.

1. Opções avaliadas

1. **React Native + Expo** (dev client + EAS Build/Update)
2. **React Native "puro"** (bare, sem Expo)
3. **Flutter** (Dart)
4. **Nativo** (Swift/SwiftUI + Kotlin/Jetpack — duas bases)

2. Comparação — dimensões técnicas

Legenda: ● forte · ● adequado · ● fraco/custoso.

Dimensão	RN+Expo	RN puro	Flutter	Nativo
Health Connect (Android)	● lib madura (react-native-health-connect) via config plugin + dev build	● idem (config manual)	● health package	● acesso 1ª classe
Apple HealthKit (iOS)	● react-native-health via config plugin	● idem	● health package	● acesso 1ª classe
Sync em background	● módulos nativos + background tasks (limites do SO)	● idem, mais config	● idem	● controle máximo
Notificações push	● Expo Notifications (APNs/FCM) + OTA	● libs manuais	● libs maduras	● nativo
Segurança	● SecureStore/Keychain/Keystore	● idem	● idem	● idem
Desempenho	● suficiente (app de dados, não gráfico)	● idem	● alto	● máximo

Dimensão	RN+Expo	RN puro	Flutter	Nativo
Manutenção	● EAS gerencia build/OTA; menos config nativa	● mais config nativa própria	● boa, mas stack isolado	● duas bases
Curva de evolução	● rápida (dev build + OTA)	● boa	● boa	● mais lenta

Nota Expo: os módulos de saúde exigem **development build** (não Expo Go) — padrão atual via `expo prebuild` + EAS, **não** é bloqueio. Expo hoje **suporta módulos nativos**; o receio antigo está superado.

3. Comparação — dimensões ESTRATÉGICAS (decisivas)

Dimensão	RN+Expo	RN puro	Flutter	Nativo
Conhecimento do time atual (Next.js/React/TS)	● mesmo idioma	● mesmo idioma	● Dart novo	● Swift+Kotlin novos
Reúso de contratos/regras/tipos da web	● alto (TS compartilhado: modelo da Observação, validação, contratos de API, cliente Supabase)	● alto	● nenhum (linguagem diferente)	● nenhum (+2 bases)
Velocidade de evolução	● alta	● média	● média	● baixa
Contratação de devs	● maior pool (JS/React)	● grande	● médio	● escasso/2 perfis
Maturidade do ecossistema	● Meta + comunidade + EAS	● Meta + comunidade	● Google	● Apple/Google
Manutenção em 5 anos	● gerenciada (OTA/EAS)	● própria	● estável, isolada	● maior custo

4. Recomendação — React Native + Expo (dev client + EAS)

Justificativa estratégica (além do técnico):

- Reúso máximo com a plataforma web:** compartilha **TypeScript** com o Next.js → um **pacote compartilhado** (monorepo) com o **modelo canônico da Observação** (HIP-007), validações, **contratos de API**, cliente Supabase e **regras de negócio**. Flutter/Nativo = **zero reúso** (linguagem diferente) e risco de divergência de contrato.
- Time e contratação:** o time já domina React/TS; maior pool de contratação; menor troca de contexto → **velocidade**.

3. **Manutenção 5 anos:** EAS gerencia builds/OTA e reduz a carga nativa; ecossistema mantido pela Meta + comunidade.
4. **Capacidade nativa suficiente:** Health Connect/HealthKit via config plugins + dev build; notificações via Expo; background workável (limites do SO valem para todos). **Trade-off aceito:** desempenho bruto inferior a Flutter/Nativo — **irrelevante** para um app de dados/saúde (sem gráficos pesados/tempo real crítico). Se algum módulo exigir performance nativa, RN permite **módulo nativo pontual**. **Descartados:** Flutter (isola o stack, sem reúso da web); Nativo (2 bases, evolução/manutenção/contratação caras); RN puro (perde os ganhos de manutenção/OTA do Expo sem vantagem estratégica).

5. O app como PRODUTO PRINCIPAL (não coletor) — arquitetura evolutiva

Estrutura pensada para **crescer sem reconstrução**, sob ARCH-002 (mobile-first · API-first):

- **Monorepo com compartilhamento MÁXIMO** (`@sintera/core` e afins) web↔mobile — além de contratos de API e tipos, compartilhar sempre que fizer sentido: **modelos de domínio** (Observação — HIP-007), **validações**, **regras de negócio**, **cliente de API** e **componentes reutilizáveis**. Fonte única → evolução sincronizada, duplicação mínima.
- **API-first:** o app consome as APIs versionadas do backend; a **web consome as MESMAS APIs** (sem fluxo exclusivo de navegador). Nenhuma regra de negócio duplicada entre plataformas.
- **Identidade única:** Supabase Auth (mesma conta) → sessão compartilhável e deep links web↔app.
- **Camadas do app:** (a) **auth/sessão**; (b) **camada de dados/offline** (cache local + fila de envio); (c) **módulo health-sync** (Apple Health/Health Connect → Observação → envio); (d) **módulos de feature** plugáveis.
- **Encaixes previstos (MVP não implementa, arquitetura acomoda) — evolução para produto completo:** **Timeline de Saúde**, **Exames e Documentos**, **Agenda**, **Medicamentos**, **Suplementos**, **notificações inteligentes** (`notif_001_infraestrutura_unica`), **captura de documentos**, **upload de exames**, **OCR**, **compartilhamento de informações** (`care_001_espaco_colaborativo`) e **IA contextual** (`visao_sistema_cognitivo_clinico`) — reusando os pilares existentes (`hub_001_registration_hub`). Nenhuma dessas exige reconstrução arquitetural.
- **Missão do MVP:** autenticar · conectar Apple Health · conectar Health Connect · sincronizar (via a arquitetura da Etapa 3). Todo o resto evolui sobre a mesma base.

6. Riscos e mitigações

- **Módulos de saúde exigem dev build** (não Expo Go) → padrão via EAS; documentar o fluxo de build.
- **Background sync tem limites de SO** → cadência realista definida na Etapa 3.
- **Revisão nas lojas** (HealthKit exige justificativa de uso + política de privacidade) → preparar no MVP.
- **Divergência de contrato web/app** → mitigada pelo pacote compartilhado (fonte única de tipos/regras).

7. Decisões pendentes da fundadora

- **D-STACK:** aprovar **React Native + Expo** (recomendado).
- **D-PLAT:** MVP primeiro em **Android/Health Connect** ou **iOS/Apple HealthKit** (pela base das usuárias do beta).
- **Contas:** Apple Developer (US\$99/ano) + Google Play Console (US\$25) + 1 dispositivo de cada.
- **Monorepo:** aprovar a estratégia de pacote compartilhado web↔app. Após aprovação, seguimos para a **Etapa 3 (Arquitetura de Sincronização)** — pré-requisito de qualquer código.